DOCUMENTATION TECHNIQUE

Projet Utribu

Module d'Administration — Plateforme Réseau Social Scolaire

1. Présentation Générale

1.1 Enjeux

Le projet Utribu est une plateforme de type réseau social conçue spécifiquement pour l'écosystème d'un micro-lycée. Dans un environnement pédagogique à effectifs réduits et à forte cohésion, l'enjeu principal est de digitaliser les interactions tout en préservant l'esprit communautaire propre à ce type d'établissement.

Les enjeux stratégiques sont les suivants :

- **Cohésion communautaire :** Créer un espace numérique dédié où élèves, enseignants et personnels peuvent interagir, favorisant ainsi le sentiment d'appartenance à l'établissement.
- **Centralisation des flux :** Regrouper les communications et la gestion de la vie scolaire au sein d'une interface unique pour éviter la dispersion des informations.
- **Sécurité et contrôle :** Garantir la fiabilité des échanges en assurant que chaque membre de la communauté est identifié et légitime, tout en protégeant les données personnelles des usagers.
- **Scalabilité :** Proposer une architecture modulaire capable d'évoluer, permettant d'ajouter de nouvelles fonctionnalités (gestion de groupes, espaces de ressources, calendrier, etc.) sans compromettre la stabilité de l'application.

1.2 Missions

L'objectif de l'intervention technique a été de développer le Module d'Administration de la plateforme. Cette mission a consisté à concevoir les outils nécessaires pour structurer et modérer la communauté naissante. Les axes de développement ont été les suivants :

- **Interface d'administration sécurisée :** Développement d'un module d'accès restreint (Drawer dynamique) visible uniquement par les utilisateurs disposant des privilèges ROLE_SUPER_ADMIN.
- **Workflow de modération des comptes :** Mise en place d'un système de validation permettant aux administrateurs de modérer les inscriptions en attente pour garantir la sécurité du réseau.
- **Gestion granulaire des rôles :** Développement d'une interface de pilotage permettant de basculer dynamiquement les droits des utilisateurs (Admin vs User), offrant une flexibilité totale dans la gestion de la communauté.
- **Cohérence UI/UX :** Intégration transparente de ces nouveaux outils au sein de l'architecture Flutter existante, en respectant rigoureusement la charte graphique et l'expérience utilisateur globale.
- **Interfaçage full-stack :** Mise en œuvre de la communication entre le backend (Spring Boot) et le frontend (Flutter) pour assurer la persistance et la réactivité des actions administratives.

2. Stack Technologique

Le projet s'appuie sur des technologies robustes, choisies pour leur complémentarité et leur performance en environnement de production.

Frontend

- **Framework :** Flutter (Dart)

- **Gestion d'état :** Provider
- **Architecture UI :** Architecture modularisée par fonctionnalités (lib/admin/, lib/auth/, etc.)

Backend

- **Framework :** Spring Boot (Java)
- **Architecture :** REST API
- **Gestion des utilisateurs :** Keycloak (OpenID Connect)

Infrastructure et Persistance

- **Base de données :** MongoDB (NoSQL)
- **Conteneurisation :** Docker et Docker Compose
- **Déploiement :** Environnement distribué (Dev local / Prod)

3. Architecture Logicielle

Le système adopte une architecture Client-Serveur découplée, favorisant la maintenabilité et l'évolution indépendante des deux couches.

Côté Backend

Une architecture en couches (Controller → Service → Repository) permet de séparer la logique métier de la persistance des données. L'accès aux ressources est protégé par des filtres de sécurité exploitant les tokens JWT fournis par Keycloak.

Côté Frontend

L'architecture suit une séparation stricte des responsabilités :

- **Services :** Couche dédiée aux appels API (ex. : admin_api_service.dart).
- **Views :** Composants UI responsables de l'affichage.
- **Modules Admin :** Une structure spécifique (lib/admin/) a été mise en place pour isoler les fonctionnalités critiques d'administration, garantissant que le code sensible n'est accessible et rendu que si les conditions de rôle (ROLE_SUPER_ADMIN) sont remplies.

Communication

Les échanges sont normalisés via le format JSON, avec une injection systématique des tokens JWT dans les headers des requêtes pour chaque appel aux routes sécurisées.

4. API Développée

4.1 Rôle de l'API REST

L'API REST constitue le point d'entrée unique pour toutes les opérations d'administration de la plateforme Utribu. Conçue selon les principes d'architecture sans état (stateless), elle expose des endpoints sécurisés permettant la gestion complète du cycle de vie des utilisateurs et de leurs privilèges. Chaque requête est autonome et contient l'ensemble des informations nécessaires à son traitement (token JWT, paramètres, payload). Les réponses sont systématiquement formatées en JSON pour assurer l'interopérabilité avec le frontend Flutter.

L'API s'appuie sur Spring Security et l'intégration OAuth2 Resource Server pour valider les tokens émis par Keycloak, garantissant que seuls les utilisateurs authentifiés et autorisés peuvent accéder aux ressources protégées. Le contrôle d'accès basé sur les rôles (RBAC) est appliqué au niveau des contrôleurs via les annotations de sécurité.

4.2 Routes d'Administration Développées

Le module d'administration expose les endpoints suivants, chacun répondant à un besoin opérationnel précis :

Méthode	Endpoint	Description	Rôle Requis
GET	`/accounts/pending`	Récupère la liste des utilisateurs ayant validé leur email mais n'ayant pas encore de rôle assigné. Interroge Keycloak pour identifier les comptes en attente de modération.	`ROLE_SUPER_ADMIN`
PUT	`/accounts/{id}/accept`	Valide le compte d'un utilisateur en attente en lui attribuant le rôle ROLE_USER. Cette opération marque la fin du processus d'inscription.	`ROLE_SUPER_ADMIN`
GET	`/users/findAllUsers`	Récupère la liste complète des utilisateurs validés présents dans MongoDB. Retourne un objet UserDTO enrichi contenant le champ roles, évitant des appels supplémentaires à Keycloak.	`ROLE_USER`
GET	`/users/findOne/{username}`	Recherche un ou plusieurs utilisateurs par leur nom (recherche par pattern). Retourne un	`ROLE_USER`

Méthode	Endpoint	Description	Rôle Requis
		UserDTO incluant les rôles Keycloak.	
PUT	/accounts/{id}/roles/admin	Promeut un utilisateur au rang d'administrateur (assignation de ROLE_ADMIN, suppression de ROLE_USER). Assure l'exclusivité des rôles opérationnels.	ROLE_SUPER_ADMIN
PUT	/accounts/{id}/roles/user	Rétrograde un administrateur au rang d'utilisateur standard (assignation de ROLE_USER, suppression de ROLE_ADMIN). Permet de révoquer les privilèges tout en conservant l'accès à la plateforme.	ROLE_SUPER_ADMIN

4.3 Composants Techniques Clés

Services Backend

- **KeycloakService :** Service centralisé responsable de toutes les interactions avec l'API Admin de Keycloak. Il expose les méthodes getPendingUsers(), acceptUser(), changeToAdmin(), changeToUser() et getUserRoles().
- **UserRepository :** Interface MongoDB permettant l'accès aux données utilisateur persistées. Utilisée en complément de Keycloak pour récupérer les informations métier (biographie, groupes, etc.).

Data Transfer Objects (DTO)

- **UserDTO :** Objet de transfert créé pour enrichir les réponses des routes /users/findAllUsers et /users/findOne/{username}. Il encapsule les attributs de l'entité User ainsi qu'une liste roles (List<String>) contenant les rôles Keycloak. Cette approche évite l'exposition directe de l'entité MongoDB et permet une transformation contrôlée des données avant leur sérialisation JSON.

Configuration de Sécurité

- **SecurityConfiguration :** Classe de configuration Spring Security définissant les règles d'accès aux endpoints. Les routes d'administration (/accounts/pending, /accounts/{id}/accept, /accounts/{id}/roles/*) sont explicitement protégées par le rôle ROLE_SUPER_ADMIN via les matchers de requêtes HTTP.

4.4 Optimisations Appliquées

Lors du développement de la route /accounts/pending, une problématique de performance a été identifiée : l'approche initiale, consistant à interroger Keycloak pour chaque utilisateur afin

de vérifier ses rôles, générait un nombre excessif de requêtes réseau, provoquant des timeouts serveur.

L'implémentation a été optimisée en inversant la logique : plutôt que de filtrer les utilisateurs individuellement, la méthode getPendingUsers() récupère d'abord l'ensemble des utilisateurs possédant un rôle (ROLE_USER, ROLE_ADMIN, ROLE_SUPER_ADMIN) via les méthodes getRoleUserMembers() de l'API Keycloak, puis effectue un filtrage en mémoire. Cette approche réduit le nombre de requêtes réseau à un nombre constant (3 requêtes au lieu de N+1), améliorant drastiquement les temps de réponse.

4.5 Intégration avec le Frontend

L'ensemble de ces routes est consommé par le service AdminApiService côté Flutter, qui encapsule la logique de communication HTTP et gère l'injection des tokens JWT dans les headers. Les réponses JSON sont désérialisées en modèles Dart (PendingUser, ValidatedUser) pour alimenter les vues d'administration.

Le champ roles nouvellement ajouté dans les réponses des routes /users/* permet au frontend d'afficher en temps réel le statut de chaque utilisateur (badge « Admin », « User », etc.) sans nécessiter de requêtes supplémentaires, optimisant ainsi la réactivité de l'interface.

5. Frontend Flutter

5.1 Architecture du Module

L'implémentation du module d'administration dans l'application mobile suit une architecture modulaire stricte afin de garantir la maintenabilité et la séparation des responsabilités (Separation of Concerns). Le code a été isolé dans un répertoire dédié lib/admin/, organisé en deux sous-couches principales :

- **Couche service (lib/admin/service/) :** Contient le composant AdminApiService, responsable exclusif de la communication avec le backend. Cette classe centralise la logique réseau, l'injection des tokens JWT dans les requêtes, la conversion des réponses JSON en objets métier typés (UserModel), et la gestion globale des exceptions.
- **Couche view (lib/admin/view/) :** Contient l'interface utilisateur (Widgets). Ces composants sont conçus pour se concentrer uniquement sur la présentation et les interactions. Ils consomment le service API et réagissent aux modifications d'état local pour rafraîchir l'écran.

5.2 Gestion de l'UI et Navigation

La navigation vers les outils d'administration est régie par des principes de contrôle d'accès basé sur les rôles (RBAC — Role Based Access Control) :

- **AdminDrawer :** Le point d'entrée du module est un menu latéral (AdminDrawer). Au sein de la structure principale de l'application (navBar.dart), la présence de ce volet est conditionnée dynamiquement. Le code vérifie la présence du privilège ROLE_SUPER_ADMIN dans le profil utilisateur : si ce rôle est absent, le menu n'est pas inséré dans l'arbre de rendu Flutter, interdisant techniquement et visuellement tout accès non autorisé.
- **PendingUsersView :** Cette interface matérialise le workflow d'approbation. Elle liste les nouvelles inscriptions sous forme de cartes d'informations. Chaque carte expose un bouton d'action permettant de déclencher directement la route de validation (/accept). L'interface bloque temporairement les actions lors du traitement réseau pour prévenir les doubles requêtes.
- **RoleManagementView :** Véritable tableau de bord de la communauté, cette vue charge par défaut l'intégralité des utilisateurs validés. Elle intègre une barre de recherche en temps réel et présente pour chaque membre un interrupteur (Switch) permettant de promouvoir ou rétrograder un utilisateur d'un simple geste.

5.3 Intégration API et Gestion d'État

Afin de garantir une expérience utilisateur fluide, l'état de l'application et les communications réseau sont optimisés selon les stratégies suivantes :

- **Réactivité avec Provider :** L'état global de l'utilisateur connecté est géré par le package Provider (via AuthProvider). Ce système permet aux composants (comme la navBar) de s'abonner aux propriétés de l'utilisateur (comme ses rôles) et de se redessiner automatiquement en cas de mise à jour, sans logique complexe de passage de variables.
- **Mise à jour optimiste de l'UI :** Lors d'une modification de rôle, la vue met à jour instantanément le modèle utilisateur en local (via setState) avant d'avoir reçu la confirmation du serveur. Cela élimine toute latence perçue. En cas d'échec de la requête HTTP, l'état précédent est restauré de manière transparente.

- **Tri dynamique local :** L'expérience de la RoleManagementView est renforcée par un tri local (Comparator) déclenché après chaque modification d'état. Ce mécanisme assure que les utilisateurs possédant le rôle ROLE_ADMIN figurent systématiquement en haut de la liste.

5.4 Détails Techniques et Résilience

- **Cohérence graphique :** L'intégralité des éléments interactifs (AppBars, interrupteurs, boutons de validation, indicateurs de chargement, avatars) respecte la charte graphique de l'application via la couleur officielle Utribu : Color.fromARGB(255, 255, 149, 0).
- **Résilience et gestion des erreurs HTTP :** Le service AdminApiService dispose d'un intercepteur (_retryOn401) enrobant toutes les requêtes. Si une erreur 401 Unauthorized survient, le service lance silencieusement un processus de rafraîchissement du token JWT. En cas de succès, la requête initiale est rejouée, rendant l'expiration du token invisible pour l'utilisateur. Les erreurs 403 Forbidden sont proprement interceptées pour afficher des messages d'avertissement via des SnackBar.

6. Conclusion

Le développement du module d'administration de la plateforme Utribu illustre la mise en œuvre concrète d'une architecture full-stack moderne, alliant robustesse technique et cohérence fonctionnelle. Ce module constitue la colonne vertébrale de la gouvernance de la plateforme : sans lui, la promesse de sécurité et de maîtrise communautaire du réseau ne pourrait être tenue.

Sur le plan technique, les choix réalisés reflètent une volonté d'efficacité et de maintenabilité. L'optimisation du mécanisme de récupération des comptes en attente — passant d'une logique N+1 à un filtrage en mémoire à partir de trois requêtes fixes — en est l'exemple le plus significatif. De même, l'adoption de mises à jour optimistes côté Flutter démontre une attention portée à l'expérience utilisateur, en éliminant toute latence perceptible lors des actions administratives.

Sur le plan architectural, la séparation stricte des responsabilités (couche service / couche vue côté Flutter, architecture Controller → Service → Repository côté Spring Boot) garantit que le module reste évolutif et testable indépendamment du reste de la plateforme. La délégation de l'authentification et de la gestion des rôles à Keycloak, via OAuth2 et des tokens JWT, externalise la complexité sécuritaire tout en offrant un mécanisme de contrôle d'accès granulaire et éprouvé.

Perspectives d'évolution

Le module tel qu'il existe aujourd'hui répond aux besoins immédiats d'un micro-lycée. Plusieurs axes d'évolution peuvent être envisagés pour accompagner la croissance de la plateforme :

- **Traçabilité et audit :** Mise en place d'un journal des actions administratives (audit log) permettant de tracer chaque modification de rôle ou validation de compte, à des fins de transparence et de conformité RGPD.
- **Notifications automatiques :** Implémentation de notifications push ou email à destination des utilisateurs lors de la validation de leur compte ou d'un changement de privilège.
- **Gestion des groupes :** Extension du module pour gérer des groupes, des classes ou des promotions, permettant une modération fine des espaces communautaires au sein de l'établissement.
- **Interface web dédiée :** Introduction d'une interface d'administration web complémentaire à l'application mobile, destinée aux usages de bureau des personnels administratifs.

Ce module d'administration pose ainsi les fondations d'une plateforme communautaire fiable, sécurisée et prête à évoluer avec les besoins de l'établissement.